Migrations and schema application

English version of ../../database/migrations.md.

Mechanism

backend/app/schema.py has three levels:

Function	What it does
reconcile(engine)	adds missing columns (additive reconciliation)
apply_migrations(engine)	applies the ordered migration registry, recording versions in schema_migrations
ensure_schema(engine)	calls both steps; safe to run on every start

The repository also holds SQL files (db/schema.sql, backend/migrations/001_schema.up.sql, 002_seed.sql) that describe the base schema and the initial seed.

Rules

1. **Migrations go forward only.** A rollback means restoring from a backup.
2. **Idempotency.** A repeated start must change nothing; this is checked by starting the API twice in a row in CI (portability.yml, job upgrade-path).
3. **Additivity is preferred.** reconcile can add columns; dropping and renaming require an explicit migration with a rationale.
4. **The data seed is separate from the schema** (002_seed.sql, AEZA_SEED_DEMO_DATA): demo data must not be a mandatory consequence of a migration.
5. **No "manual edits in production".** If something must change, it is a migration in the repository.

Upgrading an installed instance

1. make backup.
2. Stop the stack, pull the pinned image versions.
3. Start it: the application applies the missing schema steps.
4. make smoke-test, then check a few calculations and the snapshot.

Verification

Claim	Command	CI
The schema applies to an empty database	make compose-up + make smoke-test	portability.yml → compose-smoke
A repeated start is safe	two API starts in a row	portability.yml → upgrade-path
A backup can be restored	make backup → make restore → make smoke-test	portability.yml → backup-restore